Yenepoya Institute of
Arts, Science, Commerce
and Management

Project:Low Level Design
on
END TO END ENCRYPTION FOR MULTICLOUD APPS

Student Name:
Nandana EC

Industry Mentor:
Mr. Shashank

Guided by:
Ms. Vijayalaxmi

# Table of Contents


1 **Introduction** 2

1.1 Scope of the Document . . . . . 2

1.2 IntendedAudience . . . . . 2

1.3 SystemOverview . . . . . 3

2 **System Design** 4

2.1 ApplicationDesign . . . . . 4

2.2 ProcessFlow . . . . . 4

2.3 InformationFlow . . . . . 5

2.4 Components Design . . . . . 5

2.5 Key Design Considerations . . . . . 6

2.6 APICatalogue . . . . . 6

3 **DataDesign** 7

3.1 DataModel . . . . . Data . . . . . 7

3.2 Access Mechanism Data . . . . . 7

3.3 Retention Policies . . . . . 8

3.4 DataMigration . . . . . 8

4 **Interfaces** 8

5 **StateandSessionManagement** 9

6 **Caching Non-** 9

7 **FunctionalRequirements** 9

7.1 Security Aspects . . . . . 9

7.2 Performance Aspects . . . . . 9

8 **References** 10

# 1. Introduction

The Low-Level Design (LLD) for the End-to-End Encryption (E2EE) system in a multi-cloud application provides a detailed blueprint of how secure data transmission and storage are implemented across distributed cloud environments. In a multi-cloud architecture, where services and data are spread across multiple cloud providers, ensuring confidentiality, integrity, and secure access becomes a critical requirement. This document focuses on the technical design and implementation of encryption mechanisms that protect data throughout its lifecycle—from the point of origin to the final destination—without exposing sensitive information to intermediate systems or cloud providers.

Additionally, the document describes data flow across services, API-level security enforcement,
and strategies for handling failures, key rotation, and scalability in a distributed environment. By defining clear interfaces, responsibilities, and interaction patterns between components, this LLD ensures a consistent and secure implementation of encryption across all cloud environments. Ultimately, it serves as a comprehensive guide for developers to build a resilient, secure, and compliant multi-cloud system with robust end-to-end encryption capabilities.

## 1.1 ScopeoftheDocument

This document defines the Low-Level Design (LLD) for implementing End-to-End Encryption (E2EE) within a multi-cloud application environment. The scope includes the detailed design of all components responsible for securing data across multiple cloud platforms, ensuring that sensitive information remains encrypted during transmission, processing, and storage. It focuses on the internal structure, behavior, and interaction of modules involved in encryption, decryption, key management, and secure communication.

## 1.2 Intended Audience

- Developers
- System Architects/Tech Leads
- Security Engineers
- Project Managers
- Compliance Audit Teams
- QA DevOps Engineers

## 1.3 SystemOverview

**Input:**

The system accepts plaintext data from authorized client applications or services, along with user authentication credentials and access tokens. It also takes encryption parameters such as public keys, session keys, or certificates required for secure communication. Inputs may include API requests, files, or messages that need to be securely transmitted or stored across multiple cloud environments.

**output**:

The system produces encrypted data (ciphertext) that is securely transmitted or stored across cloud platforms, ensuring that unauthorized entities cannot access the original content. At the receiving end, authorized users or services decrypt the data using appropriate private keys to retrieve the original plaintext. Additional outputs include authentication responses, encryption status messages, logs, and error reports to support monitoring and auditing of secure operations.

# 2. SystemDesign

## 2.1 Application Design

The system follows layered architecture.

| Layer Presentation | Description |
|---|---|
| Layer Application | Web/Mobile apps, User requests, API calls |
| Layer Security | Business logic, Workflow orchestration |
| Layer | E2EE, Encryption/Decryption, Key Management,Authentication and Authorization |
| Data Layer | Encrypted storage, Database, Cloud storage |
| Infrastructure Layer | Multi-cloud platforms, Networking, Deployment |

## 2.2 Process Flow

1. Client Request — User sends data from web/mobile application

↓

2. Authentication Validation — System verifies user identity and permissions

↓

3. Key Generation / Retrieval — Public key is fetched or session key is created

↓

4. Encryption (Client-Side / Gateway) — — Plaintext is encrypted into ciphertext (E2EE)

↓

5. Multi-Cloud Transmission — — Encrypted data is sent across cloud providers

↓

6. Secure Storage — — Ciphertext stored in cloud databases/storage

↓

7. Decryption (Authorized Access Only) — — Data decrypted using private key at destination

↓

8. Response to Client — — Decrypted result returned to authorized user

## 2.3 Information Flow

1. Data Generation

2. Data Capture

3. Encryption Process (E2EE)

4. Secure Transmission

5. Cloud Processing Storage 6.

Access Request Validation 7.

Decryption Process

8. Output Delivery

## 2.4 Components Design

1. Client Application Layer Web/Mobile apps that generate and send data Handles user interaction and request formation

↓

2. API Gateway / Service Interface Manages requests, routing, authentication Acts as entry point to backend services

↓

3. Encryption Decryption Module (E2EE Engine) Performs encryption before transmission Decrypts data only at authorized endpoints

↓

4. Key Management System (KMS) Generates, stores, rotates cryptographic keys Ensures secure key distribution

↓

5. Authentication Authorization Module Verifies user identity and access permissions Enforces zero-trust security model

↓

6. Multi-Cloud Storage Layer Stores encrypted data across cloud providers Ensures

redundancy
and availability

↓

7. Monitoring Logging Component Tracks encryption events, access logs, errors Supports auditing and system analysis

## 2.5 Key Design Considerations

- End-to-end encryption for full data security
- Secure key management (generation, storage, rotation)
- Zero-trust access control
- Multi-cloud compatibility
- High performance with low latency
- Data integrity and confidentiality
- Fault tolerance and high availability
- Logging and auditing for compliance

### 2.6 API Catalogue

API Catalogue is used in your kind of system, especially for a multi-cloud end-to-end encryption application. It lists all available APIs (like encryption, decryption, key management, authentication) Helps developers understand how services communicate Ensures consistency across multiple cloud services Supports easier integration, testing, and maintenance Improves documentation and governance of APIs

# 3. Data Design

## 3.1 DataModel

**Independent Variables:**

- User input data (plaintext)
- Encryption algorithms used
- Key size and type (public/private, symmetric)
- Key rotation frequency
- Authentication method (tokens, certificates)
- Number of cloud providers (multi-cloud setup)
- Network conditions (latency, bandwidth)

**Dependent Variable:**

- Encrypted data (ciphertext)
- Decryption success rate
- System performance (latency, response time)
- Security level (data confidentiality, integrity)
- System availability and reliability
- Error rates during encryption/decryption
- Data access success/failure based on authorization

## 3.2 DataAccessMechanism

```
import pandas as  pd
df = pd.read_csv("encrypted_data.csv")
```

### 3.3 Data Retention Policies

- Store data only for the required duration
- Encrypted data retained based on compliance needs
- Automatic data deletion after expiry period
- Regular key rotation for stored data
- Backup retention with encryption
- Secure data disposal (no recovery possible)
- Follow legal and regulatory requirements
- Maintain audit logs for retention activities

- 3.4 Data Migration

  – Migrate data in encrypted (ciphertext) form
  – Validate data integrity before and after migration
  – Secure key transfer or re-keying during migration
  – Use secure channels for data transfer
  – Ensure compatibility across cloud platforms
  – Minimize downtime with phased migration
  – Verify access control after migration
  – Log and audit all migration activities

## 4. Interfaces

**Inputs:**

* User data (plaintext)
* Authentication credentials (username, token, certificate)
* Encryption keys or key references
* API requests (read/write operations)

**Outputs:**

* Encrypted data (ciphertext)
* Decrypted data (for authorized users)

* Authentication/authorization responses
* Status messages (success/failure)
* Logs and audit records

## 5. State and Session Management

* Maintain minimal state (prefer stateless services)
* Store encrypted state data when required
* Use tokens to track user state
* Ensure consistency across multi-cloud services
* Handle state synchronization securely

## 6. Caching

* No caching implemented
* Real-time execution

## 7. Non-Functional Requirements

### 7.1 Security Aspects

* End-to-end encryption ensures data confidentiality and integrity
* Strong authentication and authorization for all users and services
* Secure key management (generation, storage, rotation, revocation)
* Zero-trust architecture for all multi-cloud interactions
* Secure communication using encrypted channels (HTTPS/TLS)
* Access control enforced at every layer
* Audit logs for all sensitive operations
* Protection against data breaches and unauthorized access

### 7.2 PerformanceAspects

* Low latency encryption and decryption operations
* High throughput for handling large volumes of data
* Scalable architecture to support increasing users and requests

## 8. References

* OWASP Security Guidelines for Secure Application Design
* NIST Cryptographic Standards (SP 800 Series)
* AWS Key Management Service (KMS) Documentation
* Google Cloud Security and Encryption Documentation
* Microsoft Azure Security and Encryption Overview
* Research papers on End-to-End Encryption in Distributed Systems
* Multi-Cloud Architecture Best Practices (Industry Whitepapers)
* Standard textbooks on Cryptography and Network Security